

# **Face Detection and Emotion classification**

Scantamburlo Mattia, Chinello Alessandro, Piai Luca

Gli Ultimi Sumiti

Università degli studi di Padova

Academic Year 2024-2025

# Contents

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Overview and project description</b>          | <b>3</b>  |
| 1.1      | How to compile and execute the code . . . . .    | 3         |
| 1.1.1    | Regarding the VLAB . . . . .                     | 4         |
| 1.2      | Structure of the project . . . . .               | 5         |
| 1.3      | High-level description of the approach . . . . . | 5         |
| 1.4      | Face Detection . . . . .                         | 6         |
| 1.5      | Emotion recognition . . . . .                    | 7         |
| 1.6      | Inter-Process Communication . . . . .            | 9         |
| 1.7      | How we organized the work . . . . .              | 10        |
| <b>2</b> | <b>Results, Performance and Metrics</b>          | <b>10</b> |
| 2.1      | On the test set . . . . .                        | 10        |
| 2.1.1    | Face Detection Evaluation . . . . .              | 10        |
| 2.1.2    | Emotion Recognition Evaluation . . . . .         | 11        |
| 2.1.3    | System-Level Evaluation . . . . .                | 11        |
| 2.2      | Regarding video and webcam . . . . .             | 11        |
| <b>3</b> | <b>Output Images</b>                             | <b>13</b> |
| 3.1      | Images with detections . . . . .                 | 13        |
| 3.2      | Images with no detections . . . . .              | 20        |

# 1 Overview and project description

In this report we present our solution to the project proposal by the group *Ave, OpenCV, morituri te salutant*. The goal of the project is to design a system for detecting faces in an image and classifying each of them based on the emotion.

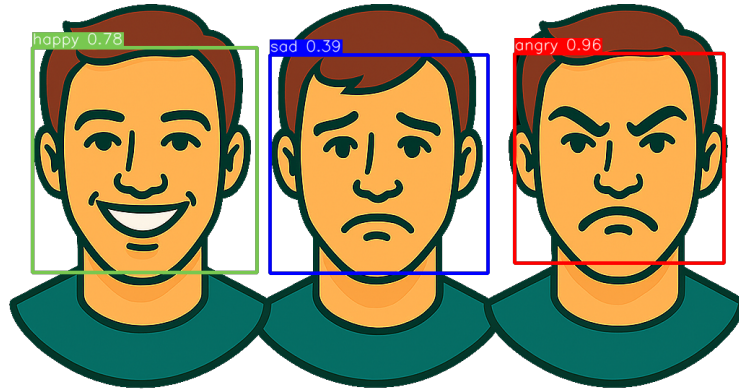


Figure 1: Logo of the project.

As suggested in the project proposal, the face recognition task is developed in C++ using OpenCV library, instead the emotion recognition is developed in Python using Keras. The entire code is open source and can be found on GitHub at the following [link](#). More in details we developed the detection pipeline exploiting the OpenCV library with the Viola Jones algorithm, while for the emotion recognition part we have fine-tuned a pre-trained **EfficientNetV2** Convolutional Neural Network.

Additionally we tried to run our pipeline even on videos and live camera videos. We added those features to our project because the pipeline that we developed to process all the images inside a given directory, was also suitable for processing images coming from a stream. We will discuss about the performances achieved in these two tasks in a dedicated subsection.

## 1.1 How to compile and execute the code

C++ dependencies:

- C++ Compiler (GCC, Clang, etc.)
- OpenCV

Python dependencies:

- Python 3.x
- Keras
- torch
- NumPy

- Matplotlib\*
- Sklearn\*
- Seaborn\*

The marked (\*) elements are necessary only if you want to train an emotion classifier using the `src/python/emotion_classifier.py` script.

To compile:

```
1 mkdir build && cd build
2 cmake .. && make
```

To execute:

```
1 Usage:
2 ./out --imgsdire <path> --labelsdir <path> --video <path> --webcam
3 Where:
4 --imgsdire ./inputs/dir -> process all images in ./inputs/dir
5 --labelsdir ./labels/dir -> labels associated to each img in ./inputs/dir (OPTIONAL)
6 --video ./video.mp4 -> process video.mp4
7 --webcam -> process webcam (default device=0)
```

For example if you want to test the program on the provided test set, you can just run:

```
1 ./out \
2 --imgsdire ../data/dataset_detection/images_scaled_max_1080 \
3 --labelsdir ../data/dataset_detection/labels
```

The images produced in output are stored under `output/detections/`, instead the metrics are saved inside `output/metrics.txt`.

Additionally you can process a video using:

```
1 ./out --video your/video/path.mp4
```

The produced video will be stored in `output/video.avi`. Note that this could take a while, depending on the video quality and frames per second.

To process the webcam stream you can use:

```
1 ./out --webcam
```

In this case will appear a window showing the output of the face detection and emotion classification in real time.

### 1.1.1 Regarding the VLAB

The C++ code compiles without error and (after installing the required Python modules) the Python script that starts the emotion classifier, runs without any issues. However we were not able to fully test the code because we do not have the permission to create fifo files. The pipeline that we have developed requires the execution of `mkfifo()` function but, since we don't have

permission to use it, the program output an error message.

**Note:** if you execute the code on the virtual machine, an output error will show up (for the reason mentioned previously) and terminates. **However** a file called `tmp/cpp_to_py.fifo` will be created. But this is NOT a fifo file and you should delete it, otherwise the next time you execute the code the program sees it as a fifo file and will crash afterwards. To execute the code without any error you must be sure that you have the right permission to create fifo files.

## 1.2 Structure of the project

The directory of the project is organized as follows:

- **data:** data used by the program. You can find the dataset used for training the emotion classifier model; the test dataset for the face detection and emotion classification part<sup>1</sup>; the Haarcascades xml files needed for Viola Jones algorithm; a pre-trained model that can be loaded with Keras modules.
- **include:** where all the header files are stored.
- **lib:** the C++ source files that implement the member functions/helper functions defined in the header files.
- **log:** where the log are stored. Here you can find `emotion_classifier_log.txt` where the standard error of the emotion classifier is redirected.
- **output:** where the final results are stored. Inside the `detections` directory you will find the input images annotated with bounding boxes and labels that indicate the predicted emotion for each detected face. If instead you have processed a video, then the resulting output `video.avi` is saved in this directory.
- **src:** inside this directory you can find `cpp/main.cpp` file, `python/emotion_classifier.py` that loads the model and predicts the emotion of the given faces and `python/trainer.py` which is the script used to train a model on the emotion recognition task.
- **tmp:** this directory stores temporary files and is used as a communication channel to share data between the C++ main program and the Python classifier. You can find the fifo files for Inter-Process Communication and the images with the cropped faces detected by Viola Jones algorithm.

## 1.3 High-level description of the approach

As already mentioned in the introduction, the application is divided in two tasks: face detection and emotion recognition. The core of the project is written in C++, instead the Python code consists in a simple script that loads a pre-trained model and classifies all the images that receives in input. The Python script is managed in a separate and concurrent thread. This tread is created at the beginning of the main function and, after loading all the necessary modules, it waits for further commands from the main thread.

---

<sup>1</sup>The results achieved are reported in the following section

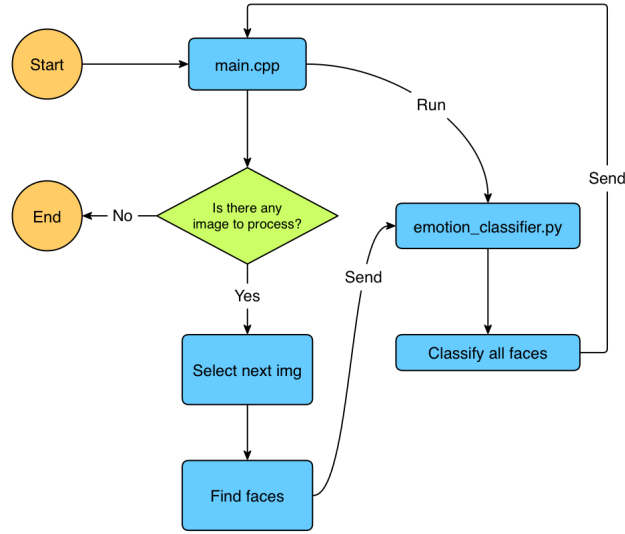


Figure 2: (Simplified) Flow diagram.

Listing 1: main.cpp

```

1 thread = run("emotion_classifier.py")
2 images_list = list of images to process
3
4 for each img in images_list:
5     faces = detect_faces(img)
6     store_to_disk(faces)
7     send_msg("start") # To Python
8     emotions = receive_emotions() # From
9         Python
10    add_boxes(img, faces)
11    write_emotions(img, emotions)
12
13 send_msg("close") # To Python.
14 thread.join() # Wait thread.
15
16 return

```

Listing 2: emotion\_classifier.py

```

1 model = load_model()
2
3 while (1):
4     msg = receive_msg() # From C++
5     if msg == "exit":
6         return # Quit
7     faces = read_disk(faces) # Collect
8         faces
9     for each face in faces:
10        emotion = model.predict(face)
11        send_msg(emotion)

```

## 1.4 Face Detection

The detection component of our program was designed in a such way to be efficient. While YOLO provides state-of-the-art accuracy, it is computationally demanding and requires powerful hardware to run effectively. For our implementation, we instead adopted the Viola–Jones algorithm, a lightweight detection framework introduced in 2001. Although it is considerably less accurate than modern deep learning methods, it is computationally inexpensive and can run in real time even on standard CPUs, thanks to its use of Haar-like features and cascade classifiers. This made it suitable for real-time webcam detection in our setup.

Nevertheless, the initial performance of Viola-Jones was unsatisfactory, as the results are highly sensitive to parameter tuning, which required careful adjustment. The main parameters taken by the Viola Jones classifier object (`cv::CascadeClassifier`) are:

- The image frame
- Level Weights
- Scale factor
- Min neighbours
- Min size
- Max size

The image was converted into a one channel grey image, and it was equalized using histogram equalization to make it easier to detect. The Level Weights is a vector representing the level of certainty with which the function recognized faces. The scale factor is the factor with which the algorithm reduces in size the image for each detection iteration (a value of 1.05 for example means the image will be reduced by 5% at each iteration). Min neighbour's parameter is the minimum number of windows in which the detection has been done, the higher this value is the more the algorithm need approval for the detection. Min size is the smaller dimension valid for a face detection. Max size is the bigger dimension valid for a face detection. The settings were chosen to be the most adaptive. Specifically, we computed the minimum size as the square root of the area divided by 150, Max size did not have a bound. Scale factor was set to 1.05 and Min neighbours to 15 as a good equilibrium of time required and precision. Also, we computed from the image the average level of Blur using the Laplacian operator. Combining this value and the area of the detected face we obtained a score representing how much the zone of the detection was usable for a good detection. Another filter to select detections was a specific minimum level of Weight for a detected face, which was specific for the used classifier. Indeed, we used two different classifiers with different Haar feature to cover more possible frontal faces.

## 1.5 Emotion recognition

Emotion recognition is achieved using Deep Learning. We decided to apply transfer learning and fine tuning to a pre-trained model. In particular we have decided to use **EfficientNetV2 B0** architecture (which is a CNN), pre-trained on **ImageNet** dataset. This architecture is fast to train and also it has less than 6 millions trainable parameters.

### Transfer learning and fine tuning approach:

1. Load a pre-trained **EfficientNetV2 B0** model.
2. Froze all layers.
3. Add some new, trainable layers on top of the frozen layers.
4. Train on our dataset (FER 2013).
5. Unfreezing the entire model<sup>2</sup>, and re-start training with a very low learning rate.

---

<sup>2</sup>All layers except for batch normalization layers. Otherwise the updates applied to the non-trainable weights will suddenly destroy what the model has learned.

The model was trained using categorical cross-entropy as loss function. The dataset that we've used is the one reported in the project proposal. Figure 3 shows the evolution of the accuracy and the loss during transfer learning (first 40 epochs) and during fine tuning (last 14 epochs). The trained model that you can find in the project repository under `/data/trained_models/` is the one that has the lowest validation loss.

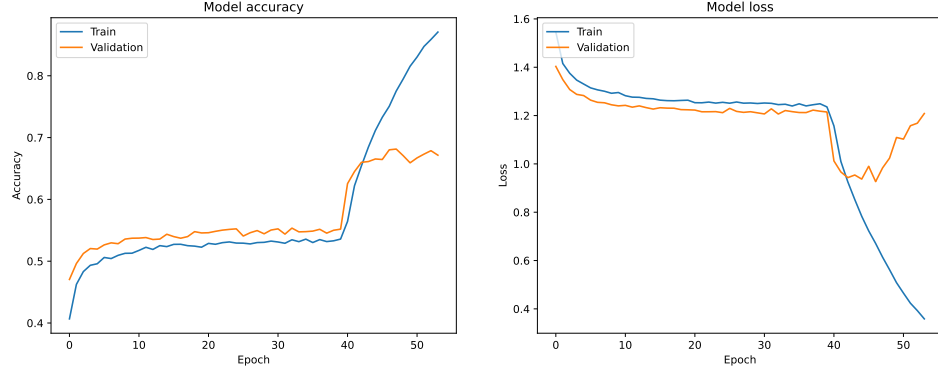


Figure 3: Accuracy and loss during transfer learning and fine tuning.

| Metric        | Value  |
|---------------|--------|
| Test Accuracy | 0.6741 |
| Test Loss     | 0.9340 |

Table 1: Accuracy and loss on the test set.

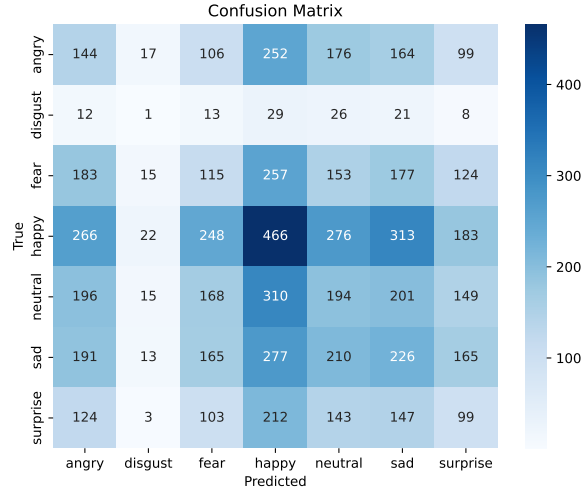


Figure 4: Confusion matrix on the test set.

Table 1 and Figure 4 show the results achieved on the test set. As we can observe, we didn't achieved high accuracy and the main reason is that classifying human emotions is not an easy tasks. Indeed, by looking at the dataset, it is not difficult to find images whose class is ambiguous to define even for us (humans).

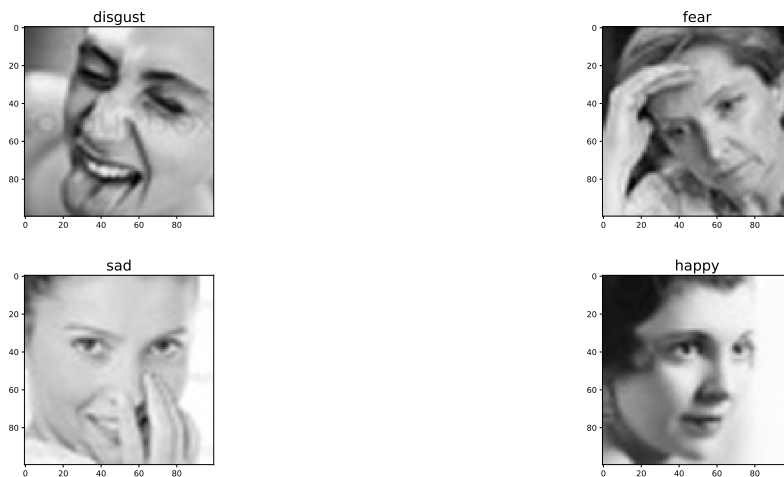


Figure 5: Selected ambiguous images.

Another issue we encountered is the class imbalance within the dataset, which is clearly visible in the confusion matrix. We attempted to use alternative datasets, but many of them either suffer from the same imbalance problem or present different limitations. However, despite this issue, as we will show in the metrics section, the classifier performs reasonably well in practice.

## 1.6 Inter-Process Communication

To connect the C++ pipeline for face detection with the Python module handling deep learning part with the fine-tuned model for emotion classification, we employed a FIFO (First In, First Out) communication system. A FIFO is a special type of named pipe used in inter-process communication (IPC).

Unlike regular (anonymous) pipes, FIFOs are associated with a special file in the file system, which allows unrelated processes to open it and exchange data. In practice, one program writes messages into the FIFO, while another one reads them in the same order (first written, first read).

This mechanism allowed us to exchange information between the C++ program and the Python program, without the need for intermediate files or complex networking protocols. In practice this mechanism has provided us a way to link the detection pipeline with the emotion recognition one.

More in details, once one image, or a frame is going to be processed we open a fifo channel named `cpp_to_py.fifo` in the cpp main file in order to send a message to the classification part that we are ready to understand the emotions of the current image/frame. Then once the face will be processed, the python pipeline will send a signal in another channel named `py_to_cpp.fifo` in a manner that is very similar to the usual semaphore mechanism that operating system adopt: when a process do its work, send a signal message to another process that is in waiting status, so it can run, and so on. This is clearly shown in the example in Listing 1 and Listing 2.

An additional important point is that this mechanism runs in a loop until the Python script receives an exit signal from the C++ program. The neural network model is loaded only once at the beginning, which avoids repeated initialization and thus improves efficiency. After that, the script remains in listening mode and processes incoming images for recognition until termination.

## 1.7 How we organized the work

The work can be divided into several temporal macro-areas, which followed one another sequentially:

- An initial phase of discussion regarding the implementation organization;
- A phase of meetings and briefings where we moved on to the actual structuring of the code, dividing the development in terms of classes and specific implementations;
- A final phase of complete revision of the structure, with the introduction of improvements and modifications.

To deliver this project we spend more or less 10 - 12 days: we organize and planned the work after the 6 of September and we start to implement the entire stack from the 8 of September finishing to clean the last details on the 18 of September. We didn't track perfectly the time spent each day, but some times we spent the entire day and other times we did less but in general we can identify a contribute of 6-7 hours per day for each component of the project, since we met working together every time on Discord as communication instrument and our virtual meeting point.

**Division of the work** Regarding the division of the files, it is difficult for us to indicate true authors since multiple people worked on the same file following a pseudo-agile approach. However a plausible division has made in the following way: Mattia Scantamburlo has implemented `face_detector.cpp`, `performance_metrics.h` and `performance_metrics.cpp`. Luca Piaia has implemented `emotion_classifier.py`, `trainer.py`, `utils.cpp`. Alessandro Chinello has implemented `main.cpp`, `utils.h`, `face_detector.h`.

## 2 Results, Performance and Metrics

### 2.1 On the test set

In this subsection we report the performances achieved on the test set that you can find in `data/dataset_detection/`.

#### 2.1.1 Face Detection Evaluation

Considering that a detection is considered correct only if the IOU computed with respect to the given labels is bigger than a threshold set to 0.4, we have the following metrics:

- True Positive TP = 39
- False Positive FP = 0
- False negative FN = 32

Metrics:

- **Precision** =  $\frac{TP}{TP + FP} = \frac{39}{39+0} = 1$
- **Recall** =  $\frac{TP}{TP + FN} = \frac{39}{39+32} \approx 0.55$
- **Average IOU of detections**  $\approx 0.65$

### 2.1.2 Emotion Recognition Evaluation

Metrics:

- **Classification Accuracy:**  $\frac{20}{40} = 0.5$
- **Confusion Matrix:**

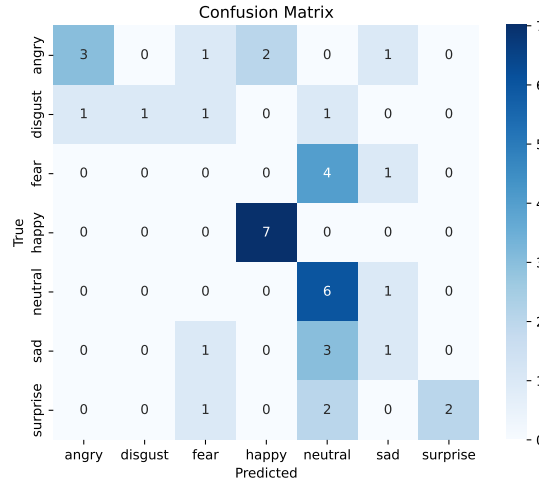


Figure 6: Confusion matrix.

### 2.1.3 System-Level Evaluation

Metrics:

- **Percentage of detected faces with correctly classified emotions:**  $\frac{20}{71} \approx 0.28$

## 2.2 Regarding video and webcam

We did not compute formal metrics to evaluate the performance on video and webcam inputs; instead, we provide some qualitative observations here.

The performance with the webcam can be smooth, depending on your machine. The parameters that we have fixed for the detection are not perfectly suited for real time processing, indeed to increase the smoothness, those parameters need to be changed. The trade of is between high detection accuracy and fast processing of a single image.

Regarding the video, the discussion is similar. Since we process each frame of the video, to get faster processing time one can reduce the number of frame per second of the input video or reduce the quality. On the GitHub repository of the project there are some examples of video produced by the program.

### 3 Output Images

#### 3.1 Images with detections

In the following we report the results that we obtained with the test set in `data/dataset_detection/images_scaled_max_1080`. Those are the images provided in the test set in the project proposal. We have rescaled them because in this way the testing process is faster and also the label with the emotion is correctly scaled. Regarding the predicted emotions, we also added a number that indicates the confidence of the emotion classifier.

Test images where at least one face was detected:



Figure 7:

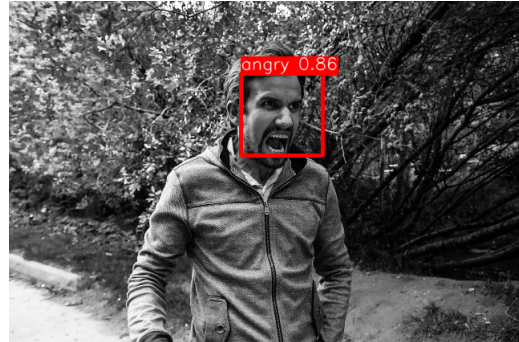


Figure 8:

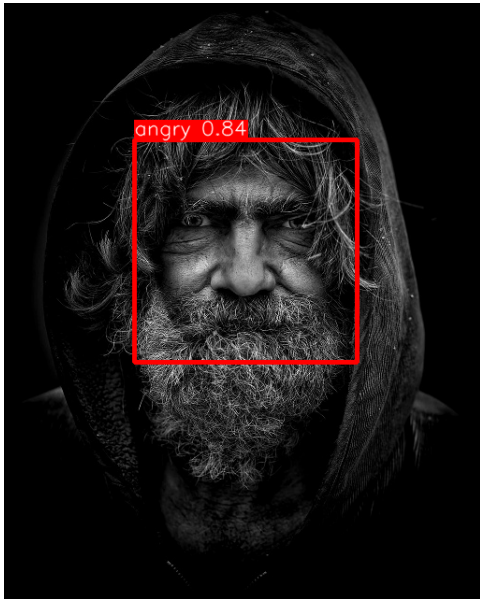


Figure 9:



Figure 10:



Figure 11:



Figure 12:



Figure 13:

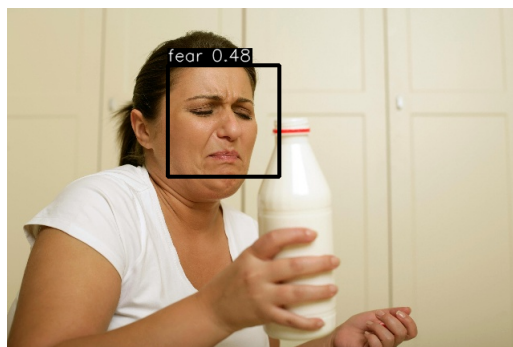


Figure 14:

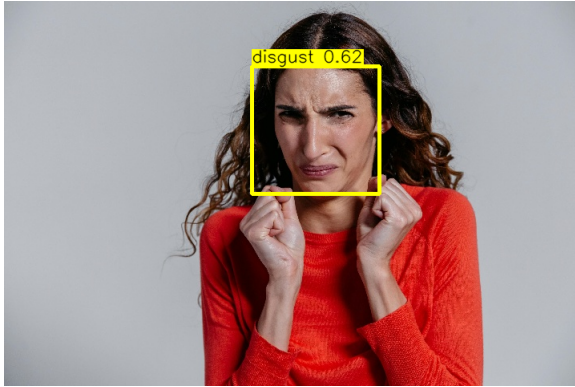


Figure 15:

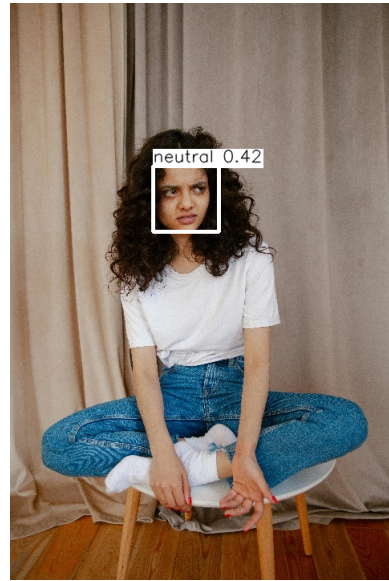


Figure 16:

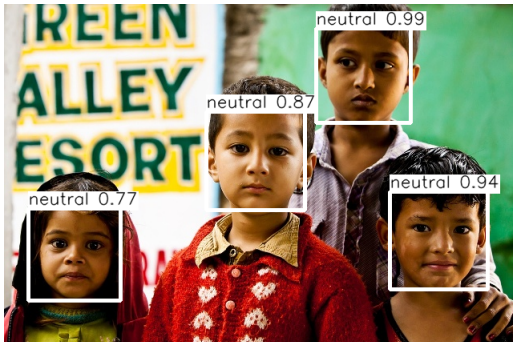


Figure 17:

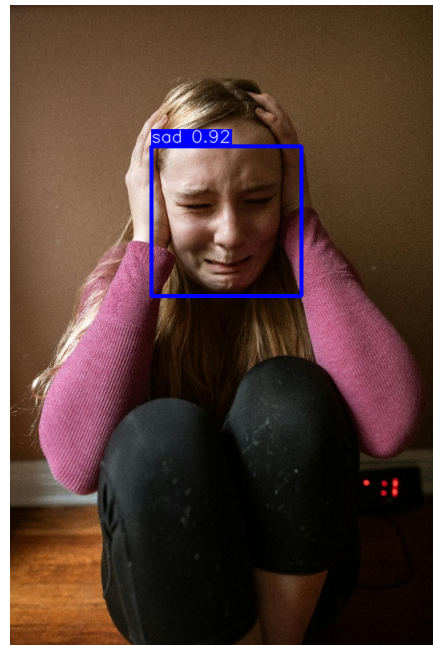


Figure 18:

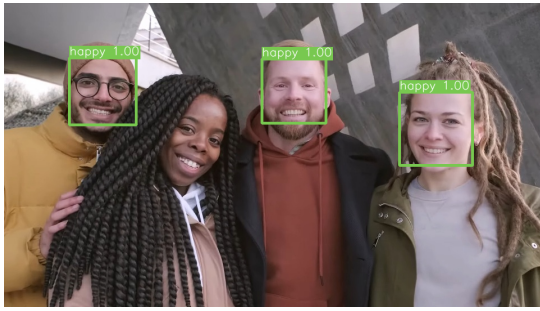


Figure 19:



Figure 20:

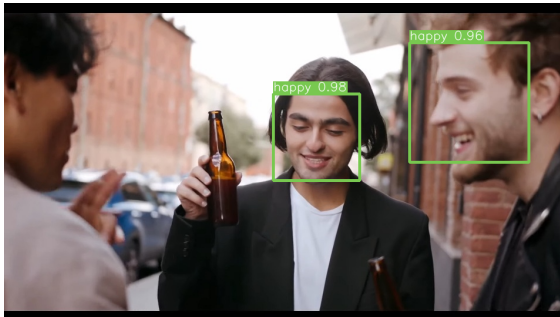


Figure 21:



Figure 22:



Figure 23:

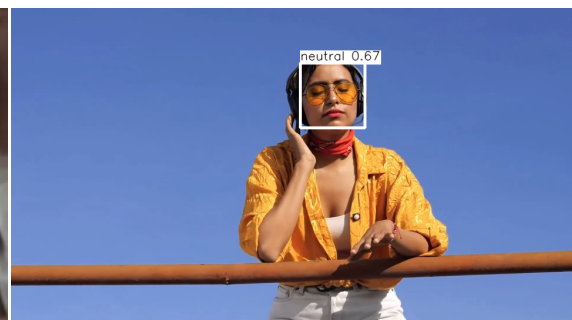


Figure 24:



Figure 25:



Figure 26:



Figure 27:

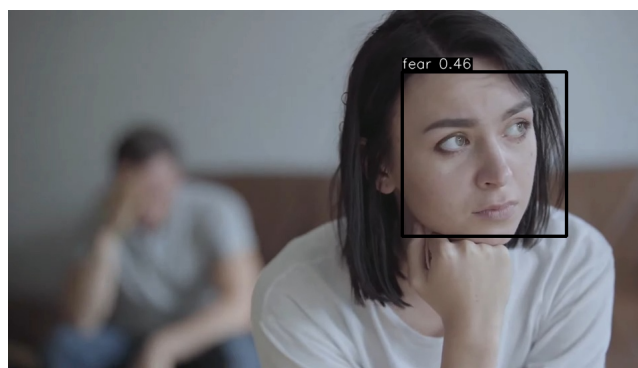


Figure 28:



Figure 29:

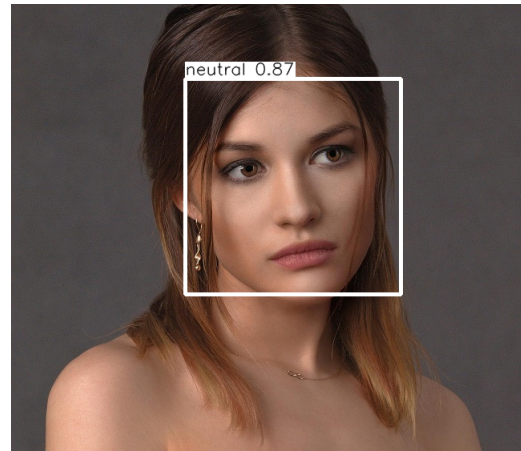


Figure 30:

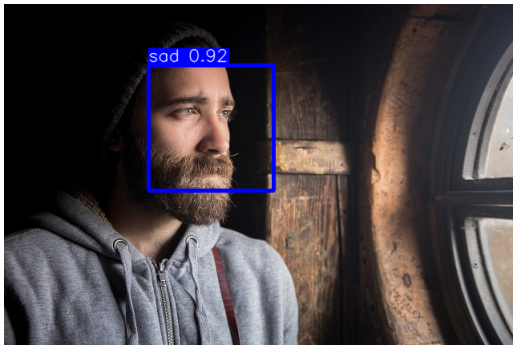


Figure 31:



Figure 32:



Figure 33:

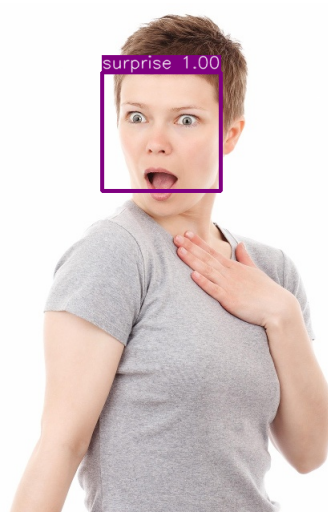


Figure 34:

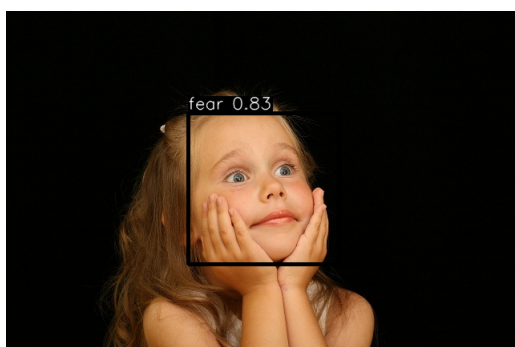


Figure 35:



Figure 36:

### 3.2 Images with no detections



Figure 37:

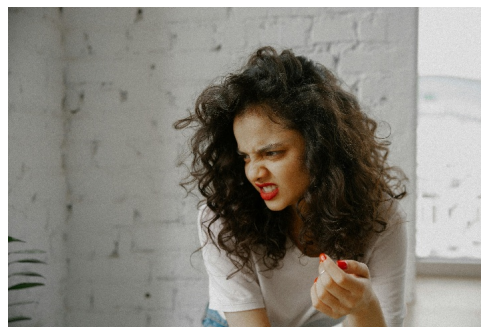


Figure 38:



Figure 39:



Figure 40:



Figure 41:



Figure 42:



Figure 43:



Figure 44:



Figure 45:



Figure 46:



Figure 47:



Figure 48:



Figure 49:



Figure 50:



Figure 51:



Figure 52: